DigitLens: A Transparent Convolutional Neural Network for Handwritten Digit Recognition

CSM422 Deep Learning, Class Project, Session 2026–27

Name: Sajal Kanwal Registration No.: 12307408

1 Objective and Scope

This project builds a small image classifier and, instead of treating the framework as a black box, re-implements its forward computation by hand and checks the two against each other. The aim is to show that the information flow, dimensionality changes and training procedure covered in the syllabus are understood well enough to be reproduced and verified, not only invoked through a library call. The project is intentionally small: one model, one dataset, one script, and no external services. It addresses CO2, CO3, CO4 and CO5; recurrent models (CO6) and the perceptron (CO1) are outside its scope.

2 Method

2.1 Data

The experiment uses the 8×8 handwritten digit set bundled with scikit-learn (1797 grayscale images, ten classes). Pixel values were scaled to [0, 1] and split 75/25 into 1347 training and 450 test images, stratified by class. A further 10% of the training images were held out by Keras as a validation set. The script also accepts a flag to load 28×28 MNIST; that path was not executed for this report.

2.2 Architecture and shape arithmetic

The network is Conv(8 filters, 3×3) → MaxPool(2) → Conv(16 filters, 3×3) → MaxPool(2) → Flatten → Dense(10, softmax), with ReLU after each convolution and same-padding. For input size n, kernel k, padding p and stride s, the output size of a convolution is

$$o = \lfloor (n + 2p - k) \,/\, s \rfloor + 1,$$

and a convolutional layer with c input channels and f filters has $(k \cdot k \cdot c + 1) \cdot f$ trainable parameters. The script evaluates these formulas and asserts that the predicted total (1898) equals the count reported by Keras.

Layer	Output shape	Parameters
Input	8 × 8 × 1	0
Conv1 + ReLU	8 × 8 × 8	80
MaxPool1	4 × 4 × 8	0
Conv2 + ReLU	4 × 4 × 16	1168
MaxPool2	2 × 2 × 16	0
Flatten	64	0
Dense + softmax	10	650
Total		1898

Table 1: Shape trace produced by the hand-written forward pass; parameter counts match the formulas above.

2.3 From-scratch layers

Convolution (with padding and stride), max pooling, ReLU and softmax were written in NumPy using explicit loops over output positions. The forward function takes the weights extracted from the trained Keras model and produces class probabilities, so that the framework result can be reproduced independently.

2.4 Training

The Keras model was trained with mini-batch gradient descent (batch size 32, learning rate 0.02, momentum 0.9) on the sparse categorical cross-entropy loss for 40 epochs with a fixed random seed. Gradients

are computed by the framework's automatic differentiation, which applies the backpropagation chain rule through every layer; backpropagation was not hand-coded here. An earlier run with learning rate 0.1 trained unstably (test accuracy 93.1%), and the rate was reduced to 0.02 before the final run reported below.

3 Results

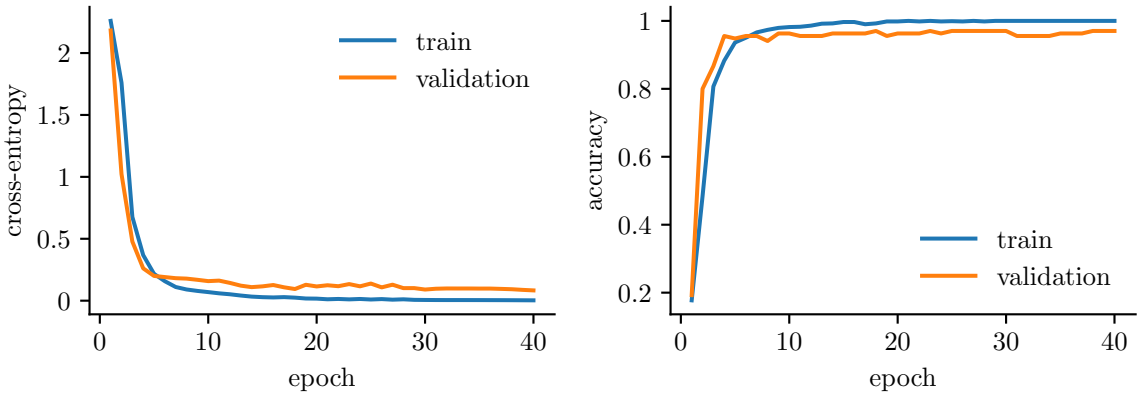


Figure 1: Training and validation loss (left) and accuracy (right) over 40 epochs.

The final model reaches a test accuracy of 98.67% (6 of 450 images misclassified) with a test loss of 0.058. The best validation accuracy was 97.0% and the final training loss was 0.003, so training loss is far below the held-out loss and some overfitting is present, which is expected for 1347 training images. Per-class test accuracy was 100% for six of the ten digits; the lowest were digit 9 (95.6%) and digit 3 (95.7%), with digit 5 at 97.8% and digit 8 at 97.7%. With so few test images per class, these per-class figures are coarse and should not be over-interpreted.

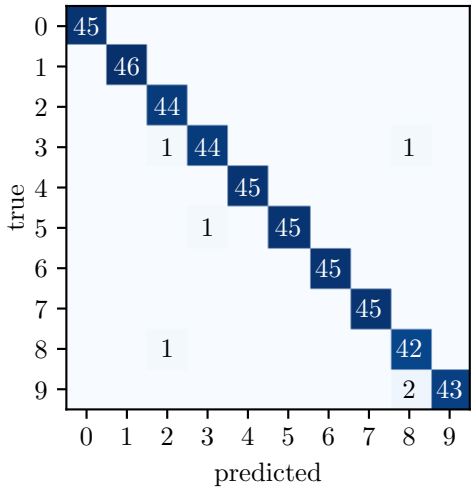


Figure 2: Confusion matrix on the 450 test images.

3.1 Verification against Keras

On the first 60 test images, the NumPy forward pass and Keras produced probabilities differing by at most roughly 0.0000002 (about two parts in ten million), and the predicted class agreed on all 60. This difference is consistent with single-precision rounding, and confirms that the shape arithmetic, padding behaviour, pooling and flattening order were understood and implemented correctly.

3.2 Inspecting what was learned

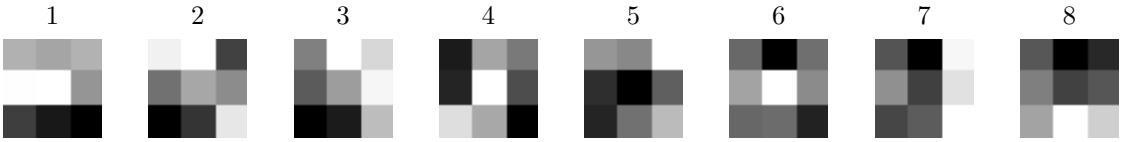


Figure 3: The eight 3×3 filters of the first convolutional layer.

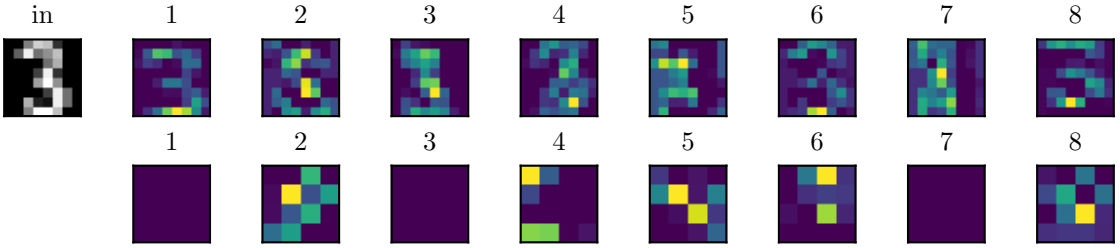


Figure 4: Feature maps for one test image of the digit 3. Top row: input and the eight first-layer maps. Bottom row: the first eight of sixteen second-layer maps (4×4).

The first-layer filters respond to local intensity patterns, and the second-layer maps are coarser (4×4) and more selective, which is the progression from local to more abstract features described in the syllabus. This is an observation from one image and a small network, not a general claim.

4 Syllabus Mapping

Outcome	Where it appears
CO2	Layer-by-layer shape trace; weight-matrix and parameter-count formulas; flattening into the dense layer.
CO3	Mini-batch gradient descent on cross-entropy loss; learning-rate adjustment after an unstable run.
CO4	Convolution, stride, padding, pooling and feature maps implemented and inspected.
CO5	CNN built, trained and evaluated in Python with Keras.

Table 2: Course outcomes addressed by the project.

5 Limitations and Extensions

The dataset is very small and low resolution, results come from a single seed, and no confidence intervals were computed. Backpropagation is delegated to the framework. Natural extensions within the syllabus are running the same script on MNIST (28×28), hand-coding the backward pass of the dense layer for comparison with autodiff, and adding data augmentation.

6 Reproducing the Results

The submission contains one script, *digit_lens.py*, which trains the model, runs the verification and writes all figures and a summary file. It requires Python, NumPy, scikit-learn, matplotlib and TensorFlow/Keras, all free to install; running it with the *–mnist* option switches to MNIST.

GitHub Link - https://github.com/Sajal-kanwal/digit-lens